TECHNICAL MANUAL · V1

Architecture & security posture

A self-hosted reverse-proxy gateway that discovers, scores, and protects APIs in-process — no traffic ever leaves the operator's own network. This document covers the architecture, the security control chain, deployment models, and an honest account of what is closed and what is still open.

LANGUAGE	CONTROLS PER REQUEST	WAF ENGINE	DEPLOYMENT
Go, single static binary	20, ordered chain	Coraza (CRS v4 opt-in)	Self-hosted, on-prem / VPC

01 · WHAT THIS IS

A gateway that sees the request before it decides whether to trust it

AEGIS is a reverse proxy: it sits in front of an existing API, forwards every request to a real backend, and — in the same process, on the same request — runs a chain of security controls and builds a live map of who calls what, with what data, and how well protected each endpoint actually is.

The problem it addresses: a signature-based WAF (SQLi/XSS pattern matching) does not understand business context. It cannot tell that a caller who normally touches three orders just enumerated three hundred, or that an endpoint returning a credit-card number has no authentication in front of it. Those are the OWASP API Top-10 classes — **BOLA, BFLA, excessive data exposure, shadow endpoints** — and they look like perfectly valid HTTP traffic to a signature engine.

SEE	PROTECT	PROVE
Passive discovery of every endpoint actually hit, path-normalized, PII-classified, scored 0–100 by posture, compared against a declared OpenAPI contract for drift.	WAF, JWT auth with signed identity propagation, rate limiting, IP/bot/threat controls, and BOLA/BFLA detection built on a consumer graph — all inline, one process.	Posture scoring, a persistent audit log, multi-tenant isolation, findings mapped to NIS2 / ISO 27001 / OWASP, exportable reports.

What makes this different from a WAF-only product

Every request that reaches a backend is attributed to a consumer identity (JWT subject, API key, or IP) and, where the response body names an object's owner, to that object. Over time this produces a **consumer graph** — not a signature list — and it is what catches the abuse a signature engine structurally cannot see: a valid, authenticated, correctly-formed request for someone else's data.

Deployment model

Single Go binary, no agents, no data plane leaving the operator's own network — the traffic AEGIS inspects never transits a third-party cloud. This targets the segment that a SaaS-only competitor (Salt, Noname, Imperva) structurally cannot serve: regulated mid-market on-prem or in a private VPC, where sending traffic to someone else's cloud is not an option.

02 · ARCHITECTURE

Two servers, one process, a 20-step ordered chain

A single binary runs two HTTP servers side by side: the **gateway** (data plane — the reverse proxy every request flows through) and the **admin API** (control plane — dashboard, findings, configuration, IAM). Config is hot-reloaded: a file change rebuilds the handler chain and swaps it atomically, so in-flight requests keep running against the old chain.

The data-plane middleware chain (source of truth: `internal/gateway/chain.go`)

Ordering is load-bearing and deliberate — e.g. tenant resolution must run first; spoofable identity headers are stripped before anything downstream trusts them; discovery wraps *around* auth/DLP (outside them in the chain) so it can enrich each observation, after the fact, with the identity and PII signals those layers attach, and outside the proxy call so it captures the final response status.

1 TenantResolve → 2 CleanHeaders → 3 UpstreamFingerprint → 4 TLSFingerprint → 5 SecurityHeaders → 6 RequestID → 7 PathSanity → 8 CORS → 9 IPGuard → 10 ThreatFeed → 11 RateLimit → 12 BotProtection → 13 Challenge → 14 WAF → 15 Discovery → 16 Auth (JWT) → 17 SchemaValidation → 18 AbuseDetection (BOLA/BFLA) → 19 DLP → 20 BehaviorAnalysis

State backends

REDIS

Rate-limit windows, IP blocklists, behaviour scores, JA3 sets, BOLA object-ownership counters, challenge tokens, JWT revocation (JTI), admin sessions, metrics, a forensic ring buffer. Most keys carry TTLs. Multi-tenant keys are prefixed `gw:t:<tenant>:*`.

POSTGRESQL **OPTIONAL**

The discovery catalog and forensic log share one database, enabled only when a DSN is configured. Without it, Discovery degrades to passthrough — the gateway still runs, it just doesn't build a catalog. Every table carries `tenant_id` with Row-Level Security as a fail-closed backstop.

Identity propagation to backends

After JWT verification, the gateway signs the forwarded identity — `HMAC-SHA256(secret, "sub:roles:scopes:ts:nonce")` — into `X-Gateway-Signature`. A backend can verify authenticity, timestamp freshness, and replay (nonce) using the reference SDK (`sdk/gatewayverify`), so a compromised edge cannot silently forge an identity a backend trusts. Inbound `X-Gateway-*` headers are stripped before this point, so a client can never inject a fake one.

03 · THE DIFFERENTIATOR

Catching the leak a signature WAF cannot see

BOLA/BFLA detection (`AbuseDetection` middleware, wired after JWT so it sees verified roles) is the primary reason a team buys API-specific security rather than "another WAF." It is built entirely on the consumer graph, not signatures.

BFLA — BROKEN FUNCTION-LEVEL AUTHORIZATION

A consumer hits a privileged path prefix without holding any of the roles that path requires. Configured per-path (`privileged` rules).

BOLA / IDOR — TWO INDEPENDENT SIGNALS

(1) **Enumeration**: distinct object-IDs per consumer/endpoint against a hard threshold and an adaptive per-consumer EWMA baseline. (2) **Single-object ownership**: learns which consumer owns which object from response bodies, flags a different consumer successfully (2xx) reading it.

Why single-object detection matters most

Enumeration catches scraping. It does not catch the case that actually leaks data in practice: one authenticated, legitimate-looking request for *one* object that belongs to someone else. AEGIS reads the declared owner field out of the response body (`owner_fields`), compares it against the authenticated caller, and — when the binding is confirmed — can **deny the next cross-owner access before it reaches the backend**, not just log it after the fact. A backend 4xx (the app already denied it) is correctly not flagged; only a successful, wrongful read counts.

> EVALUATED LIVE, NOT ASSERTED
>
> A minted-JWT walkthrough (`demo/idor-demo.sh`) drives this end to end against a deliberately vulnerable backend: a second user's order leaks once, AEGIS confirms the IDOR from the response body with an explainable `why` , and every repeat attempt is denied at the gateway — the legitimate owner is unaffected throughout.

False-positive controls

- An allowlist exempts known-benign high-cardinality callers (batch jobs, search indexers, internal admins) from enumeration detection entirely.
- Ownership-bypass roles (support/admin) are exempt from the proactive-deny path.
- The per-consumer adaptive baseline reduces false positives from callers whose normal volume is simply higher than the global threshold.

COMPLEMENTARY SIGNATURE LAYER

WAF: Coraza, OWASP CRS v4 opt-in

Alongside behavioural detection, the gateway runs Coraza — the built-in starter ruleset by default (zero-setup, dev/demo-grade), or the full **OWASP Core Rule Set v4** (~900 rules, anomaly scoring, paranoia levels 1–4) with `use_crs: true` — the production-grade choice, and the default in the Helm chart's enterprise profile. `block_mode: false` runs either ruleset in detection-only mode for tuning false positives first.

04 · SEE, THEN PROVE

Passive discovery, posture scoring, compliance mapping

Every request produces an observation; a background worker aggregates observations per 5-second window and flushes normalized deltas to PostgreSQL. Paths are normalized (`/users/42` → `/users/{id}`) so the catalog reflects real API surface, not raw URL noise.

Posture classification

POSTURE	MEANING
`protected`	Auth required and enforced, WAF/DLP active per the effective route config.
`partial`	Some controls active, not the full set the route's risk profile would call for.
`unprotected`	Endpoint is live, receiving traffic, with no meaningful controls in front of it.
`shadow`	Observed in real traffic but absent from the declared OpenAPI contract — API9.

Findings — the buyer-facing view

Findings are derived per endpoint from accumulated traffic and mapped to OWASP API Top-10: **API3** (PII returned to an anonymous or unauthenticated-in-practice caller — confirmed critical vs. latent warning depending on whether auth is merely undeclared or actually not enforced) and **API9** (a shadow endpoint that also leaks PII). Severity-sorted, critical first, exportable as JSON or CSV.

Compliance mapping

Findings and runtime abuse detections (BOLA/BFLA events) are mapped through OWASP API Top-10 onto concrete framework controls — for example an IDOR maps to **NIS2 Art. 21(2)(i)** and **ISO/IEC 27001:2022 Annex A.8.3** — with severity and the underlying issue list attached to each control. This is a pure, unit-tested mapping function, not a marketing claim layered on top after the fact.

> SCOPE, STATED PLAINLY
>
> OWASP API Top-10 coverage today: BOLA/BFLA (API1/API5) enforced, PII-exposure and shadow-endpoint findings (API3/API9) detected, mass assignment (API6) enforced via schema validation (`additionalProperties: false` rejects undocumented body fields). Injection-pattern findings and broken-auth velocity

> detection are not yet built. PCI-DSS/HIPAA/GDPR compliance templates are not yet built — NIS2/ISO 27001 only, today.

05 · HOW A TRIAL ACTUALLY WORKS

Observe mode: inline, and provably unable to break anything

The standard objection to any inline security product is "what if it blocks or breaks something in front of my production traffic." AEGIS has a deployment mode built specifically to make that objection moot for the trial period.

`observe: true` is a single top-level flag. A dedicated function (`ApplyObserveMode`) runs *after* config validation, on both startup and every hot-reload, and coerces the entire security configuration into a guaranteed non-disruptive shape — it cannot be bypassed by a per-route override or a later config edit.

CONTROL	ENFORCE MODE	OBSERVE MODE
WAF	Blocks (403)	Detection-only — matches and logs, forwards the request
DLP	Redacts PII in the response	Classifies and flags; body passes through unmodified
JWT auth	401 on missing/invalid token	Extracts identity from a valid token; otherwise passes through
BOLA/BFLA	Can proactively deny	Detect-only
Rate limit, IP guard, bot, challenge, behavior	Block / throttle / ban	Disabled entirely
fail_closed (rate-limit, IP-guard, revocation)	May deny on a Redis outage	Forced off — never fail closed

What still runs, and produces the trial's value: passive discovery, DLP *classification* (which endpoints leak which PII types), BOLA/BFLA *detection* (identity-attributed when a valid token is present), WAF detection, schema drift. A loud `OBSERVE MODE ACTIVE` warning is logged at startup and on every reload.

> VERIFIED, NOT JUST CONFIGURED
>
> A built-in WAF rule-engine asymmetry was found through this exact process: the embedded starter rules hardcode an inline `deny` under `SecRuleEngine On`, so `block_mode: false` alone did not stop them from returning 403 on a live probe. Fixed by forcing `DetectionOnly` specifically for observe mode. Caught by testing on a running binary against a deliberately maximally-blocking config, not by unit tests alone — SQLi, XSS, a missing JWT, and a rate-limit trip all now pass through as 200, while WAF/DLP/BOLA detections are still recorded.

A trial, concretely

1. Deploy in front of the partner's live traffic with `observe: true` — one YAML change, a ready-made template ships with the product.
2. Let it run for a week. Nothing is blocked, redacted, or rejected at any point.
3. Read the findings: shadow endpoints, PII exposure, confirmed BOLA/BFLA events, mapped to NIS2/ISO 27001 — as a downloadable CSV, no console access required.

Caveat, stated honestly: observe mode is still *inline* — the gateway sits in the request path and adds a proxy hop, even though it blocks nothing. True out-of-band ingestion (traffic mirroring / eBPF) removes the hop entirely and is scoped but not yet built (roadmap item B1).

06 · ISOLATION AND FAILURE BEHAVIOR

Multi-tenant by construction, tested under real failure

Tenant isolation

`TenantResolve` runs outermost in the chain — it maps a request to a tenant by route or Host, rejects a mismatch or unresolved tenant with 404, and strips client-supplied `X-Tenant-*` headers so a tenant can never be spoofed from outside. Isolation is then enforced at every storage layer:

REDIS

Every key family (rate limits, blocklists, sessions, forensic buffer, ~15 families total) is prefixed `gw:t:<tenant>:*` — cross-tenant key access is structurally impossible, not policy-enforced.

POSTGRESQL

Composite primary keys carrying `tenant_id`, every query filtered by it, **plus Row-Level Security** as a fail-closed backstop — an unscoped query returns zero rows rather than another tenant's data, even if application code forgot the filter.

Console sessions are pinned to a tenant with admin/viewer RBAC; cross-tenant deny paths (session, blocklist, metrics, forensic log) are covered by dedicated tests against live Redis and PostgreSQL.

Measured failure behavior (on-hardware, sustained traffic)

REDIS KILLED MID-TRAFFIC

100%

Requests still succeeded (fail-open by design), p99 bounded by fast-fail timeouts rather than the multi-second hangs default client timeouts would cause. Latency snapped back to normal immediately on recovery.

POSTGRESQL KILLED MID-TRAFFIC

100%

Data-plane traffic held 100% success with flat latency — the catalog/forensic write path is fully decoupled (async) from proxied requests. Only the admin catalog *read* degrades, and self-recovers.

ROLLING DEPLOY (SIGTERM)

Zero 5xx

A lame-duck drain (`/readyz` → 503 → finish in-flight → stop) behind a readiness-gated load balancer turns what used to be a burst of connection errors into a clean rollout.

Multi-tenancy performance overhead

Single-tenant vs. three-tenant mixed traffic, same VU count, same hardware: throughput moved from 373.6 to 375.7 req/s and p50 latency by roughly half a millisecond — the isolation cost of per-tenant Redis prefixes and RLS policy evaluation is in the noise, not a meaningful tax.

07 · SECURITY OF THE TOOL ITSELF

A security product that is audited the same way it audits others

Five GitHub Actions workflows run on every commit. None of this is a one-time claim — it is enforced on every push, and it has caught real bugs, not just theoretical ones.

DEPENDENCY & STDLIB CVES	STATIC SECURITY ANALYSIS	TEST COVERAGE GATE	DYNAMIC SCAN
`govulncheck`, pinned Go toolchain, every commit.	`gosec` medium+ and `golangci-lint`, every commit.	Per-package floors that only ratchet upward, against live Redis + PostgreSQL.	E2E smoke test plus a `nuclei` pass on a live admin plane.

Current coverage (against live Redis + PostgreSQL, not mocks)

`tenant / tlsfp 100% · gatewayverify 94% · classify / proxy 92% · config 90% · gateway 87% · sso 88% · discovery 84% · store / middleware 83% · iam / retention 82% · audit 81% · api 78% — every critical package at or above the 70% release floor.`

A concrete example: this month's self-audit

A full pass across the codebase (automated scanners plus manual review of everything changed that quarter) found and fixed, in order:

1. **Log injection (CWE-117)** — a client IP reached a log line without the same CR/LF stripping applied to the adjacent fields on that line. Fixed and covered with a regression test.
2. **A stdlib CVE patched in one Go module, silently unpatched in a second** — a separate module (the marketing site's tiny backend) had no pinned toolchain, so `govulncheck` found four real, reachable CVEs, including the exact TLS CVE already fixed elsewhere weeks earlier. Pinned to match.
3. **A CI blind spot** — that same second module was never scanned by any CI workflow at all, because it lives in its own `go.mod` and none of the pipelines descended into it. Closed by adding it everywhere Go checks run.
4. **An untested new endpoint** — a just-shipped admin API route had no unit test. Verified live on a running binary first (unauthenticated → 401, valid credential → correct data, invalid credential → 403), then covered.

Included here deliberately, not omitted: a security product's credibility rests on how it handles finding its own bugs, not on a claim of having none.

08 · WHERE THIS ACTUALLY STANDS

Closed, and open — without rounding up

Closed

- Console authentication: bcrypt + session cookies, CSRF, admin/viewer RBAC
- OIDC single sign-on (Authorization Code + PKCE, JIT provisioning)
- Multi-tenancy — all 6 phases (ingress, PG isolation + RLS, Redis isolation, session RBAC, tenant/user CRUD, load-tested)
- Real TLS/JA3 fingerprinting from the ClientHello, spoofable header stripped
- `fail_closed` for rate-limit and IP-guard on a Redis outage
- BOLA/BFLA detection with confirmed single-object ownership + proactive deny
- OpenAPI spec-drift detection (undocumented + zombie endpoints)
- Schema enforcement — positive security against an imported contract
- Observe/pilot mode — the guaranteed non-disruptive trial posture
- Data retention sweep, HA runbook (Sentinel + PG streaming replication)
- Prometheus metrics, admin audit log, secret-rotation runbook
- NIS2 / ISO 27001 compliance mapping

Open — stated without softening

GAP	STATUS	WHY IT MATTERS
Independent manual pentest	NOT STARTED	The one item that cannot be self-certified by any amount of internal tooling. Currently paused on budget. Does not block a trial run in observe mode — it blocks selling enforce-mode in production.
Out-of-band deployment	NOT BUILT	Traffic mirroring / eBPF ingestion, for partners who cannot add any inline hop at all. Scoped, deliberately deferred until a specific partner requires it.
Production-grade capacity number	PARTIAL	Internal benchmarking shows no measurable overhead from observe mode versus enforce mode, and WAF cost is small at moderate load — but a trustworthy absolute max-RPS figure needs dedicated server-grade hardware; current numbers are floor estimates from consumer-grade test hardware.
OWASP API Top-10 breadth	PARTIAL	BOLA/BFLA (API1/API5), PII/shadow-endpoint findings (API3/API9), and mass assignment (API6, via schema enforcement) are built. Injection-pattern findings and broken-auth velocity are not yet.

THE HONEST PITCH

This is a strong, actively-hardened implementation with a real technical differentiator (the consumer-graph-based BOLA/BFLA detection) and a genuinely safe way to trial it. It is not yet independently pentested, and it does not yet match a mature SaaS competitor's breadth. Both of those are stated here on purpose.